

# Documentação Técnica: Sistema de Recomendação de Streaming

Este documento apresenta a especificação técnica e a explicação detalhada de cada módulo desenvolvido no Sistema de Recomendação de Streaming (Trabalho Final - ED1). O projeto foi desenhado sob uma arquitetura modular, dividindo as responsabilidades em arquivos separados para garantir facilidade de manutenção e um código limpo.

O projeto aderiu fielmente ao padrão de Orientação a Objetos simplificado, o mesmo padrão observado no código de referência (ArvoreB.cpp), utilizando structs para a representação dos Nós de Dados e classes para agrupar e implementar as lógicas de manipulação.

---

## 1. Módulo Central de Tipos (conteudo.h)

O arquivo `conteudo.h` atua como o coração tipográfico do sistema. Ele concentra as definições de domínio que serão compartilhadas por todos os outros módulos (listas, árvore e estatísticas).

### 1.1 Enums (Tipos Enumerados)

- Tipo: Identifica se o conteúdo é FILME, SERIE, DOCUMENTARIO, ANIME ou OUTRO\_TIPO.
- Genero: Classifica as categorias (ACAO, COMEDIA, DRAMA, TERROR, FICCAO, OUTRO\_GENERO).
- FiltroAno: Um domínio extra para a lógica da árvore, ditando se a busca busca conteúdos de APENAS\_RECENTES ( $\geq 2021$ ) ou APENAS\_CLASSICOS ( $< 2005$ ).

### 1.2 Estruturas de Dados (Nós)

- Conteudo: Nó principal do catálogo de filmes. É o nó que compõe a *Lista Simplesmente Encadeada*. Possui campos detalhados: nome (string), as chaves de tipo, visualizações, etc. Tem seu ponteiro `prox` para o próximo elemento da lista.
- ConteudoHistorico: Nó da *Lista Duplamente Encadeada*. Focado estritamente nas métricas, contém as chaves para armazenar os mais assistidos. Tem os ponteiros `prox` e `ant`.
- NodoDecisao: Nó da *Árvore Binária de Decisão*. Armazena a pergunta (texto) e aponta para a resposta afirmativa (`esq`) e negativa (`dir`). Caso seja o nó terminal (folha), armazena os critérios extraídos dessa trilha de decisão.

### 1.3 Funções Auxiliares

Além das tipagens, o arquivo engloba conversores que traduzem os Enums numéricos para Strings textuais legíveis para apresentação ao usuário (exemplo: `tipoParaString()`), bem como leituras interativas validadas (`LerGenero()`).

---

## 2. Módulo Catalogo (lista\_simples.h / lista\_simples.cpp)

A classe ListaSimples é o repositório principal dos conteúdos. Trata-se de uma **Lista Simplesmente Encadeada e Ordenada**.

- **Funcionamento de Inserção:** O método `inserirOrdenado()` jamais insere os nós no final. A cada nova solicitação, a lista é inteiramente percorrida a partir do ponteiro inicio. Usando a verificação textual < da string, a classe garante que a listagem de filmes esteja perfeitamente organizada em ordem ALFABÉTICA antes de quebrar as correntes (ponteiros) e inserir a nova struct adequadamente;
- **Remoção Iterativa:** Por meio do `removerPorIndice()`, a lista consegue saltar  $N-1$  elos com um loop limitando o avanço. Quando atinge o predecessor, descola o nó procurado ligando o predecessor diretamente ao sucessor, libertando a memória (delete) do escolhido de forma segura (Semanticamente: Livrando-se de Memory Leaks nas remoções).
- **Filtros e Mecanismo de Busca (listarFiltrado):** O Listar interage com as propriedades do Conteúdo. Se a Árvore exigir Dramas, ele irá desconsiderar os Filmes de Terror, validando a cada pulo o critério de match na exibição.

---

## 3. Módulo de Histórico (lista\_dupla.h / lista\_dupla.cpp)

A classe ListaDupla resolve a premissa de armazenar “os títulos mais assistidos”. Diferentemente do catálogo que baseia sua ordem alfabeticamente, o Histórico subverte essa regra implementando foco absoluto no elemento “Visualizações” operando sob **Ordem Decrescente**.

- **Funcionamento do `inserirOuAtualizar()`:** É um algoritmo altamente complexo de reordenação dinâmica (Self-Balancing behavior). Uma vez que um usuário assiste a um filme pela plataforma (Opção 4 do Menu).
  1. O método busca se a chave (nome) já existe. Se não existir, constrói um nó padrão (Veze Assistido: 1) no fundo da lista duplamente encadeada.
  2. Se a chave existir, ele incrementa a visualização (+1).
  3. Imediatamente após incrementar, recorta (Desencadeia) aquele filme do nó (`ant->prox = prox; prox->ant = ant`).
  4. Recomeça sua busca de cima (do inicio), e reposiciona este nó superior de acordo com a sua nova pontuação, escalando-o em direção ao topo; garantindo que essa lista mostrada ao usuário reflita o tempo real os títulos que o povo mais consome.

---

## 4. Módulo “Machine Learning” (arvore\_decisao.h / .cpp)

A classe ArvoreDecisao constrói o sistema central base das interações

comportamentais utilizando a estrutura recursiva de **Árvores Binárias**.

- **Arquitetura (Mínimo de 6 Níveis Garantido)**: A estrutura possui 5 perguntas ramificando no lado mais profundo (que vão se traduzir na folha de índice Nível 6). A base começa generalista (Deseja série ou filme? -> Gosta de Ação? -> Prefere Lançamentos Recentes?).
  - **Lógica Recursiva**: A função `navegar()` é chamada referenciando a raiz, exibe o atributo pergunta em tela. O input do usuário dita se a chamada recursiva irá passar no parâmetro o ponteiro da Esquerda (Equivalente ao Sim) ou viaja pela Direita (Equivalente ao Não). Conforme avança na profundidade, quando choca em um nó onde `ehFolha == true`, a recursão quebra, devolvendo - **por Referência de Ponteiros Estáticos (&tipo, &genero)** - as características matemáticas do conteúdo filtrado ao sistema principal.
  - **Impressão (2D)**: Baseado inteiramente no arquivo de referência, roda um sistema de percorrimento "Direita-Raiz-Esquerda", usando espaços horizontais atrelados a profundidade para imprimir graficamente como o sistema decide nas escolhas.
- 

## 5. Módulo Estatístico e Extra (estatisticas.h / .cpp)

A classe Estatisticas trabalha unificando os comportamentos das listas para extrair as matrizes solicitadas pela Professora Maria Inés e entrega da **Funcionalidade Extra**.

- **Métrica Cruzada**: Arrays (Contadores) foram instituídos `contadorTipo[5]` e `contadorGenero[5]`. Para cada recomendação bem sucedida, ele infla estes loops. A varredura calcula o Maior (Elemento mais Recomendado) cruzando logicamente esses números.
  - **Funcionalidade Extra: AVALIAÇÃO DE ESTRELAS**: Adicionou as variáveis float nas raízes dos nós, implementando a matemática Média Ponderada onde cada avaliação altera o float para um teto estrito 1.0-5.0, imprimindo uma progressão linear gráfica em tela com caracteres unicode ★ se completas, e ☆ em caso contrário. É possível extrair um Ranking Ordenado.
- 

## 6. Integração Final e Interface (main.cpp)

O arquivo primordial funciona sob o laço `for (for(;;))` iterativo de interrupção manual via opção 0).

- Ele instância todos os objetos: `ListaSimples catalogo`, `ArvoreDecisao arvore`, etc.
- Atua com **Hardcode Injector** que cria (Mock) os primeiros 20 filmes do sistema.
- É responsabilidade do main engatar a função `limparTela()` (Baseada em MACROS do Pre-Processador em runtime se chama CLS ou CLEAR) garantindo a legibilidade.
- **Fluxo (Core Experience)**: Ele é o canal que chama a *Arvore* para decidir, captura a refutação enviada pela folha da Arvore, transmite para a *ListaSimples* para que encontre o filme e ao final, envia para a *Lista Dupla* para atualizar o

histórico.